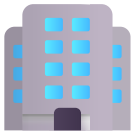


Building Manager Network

Comprehensive Business & Technical Project Analysis

Document Type: Project Analysis Report

Date: July 2026

Version: 1.0

Classification: Internal

Table of Contents

1. Executive Summary

The **Building Manager Network** is a full-stack, production-grade platform designed to digitize and streamline the management of multi-tenant residential and commercial buildings. It serves as a unified ecosystem connecting building managers, property owners, tenants, property agents, and professional service providers.

> **Vision:** To become the leading digital infrastructure for building management, transforming how common expenses, maintenance requests, and stakeholder communication are handled in multi-tenant properties.

Key Highlights

- **Domain:** Property Technology (PropTech) — Building & Community Management
- **Architecture:** Monolithic Spring Boot REST API with Angular frontend (separate repository)
- **Authentication:** JWT-based with email verification, password reset, and role-based authorization
- **Core Functionality:** Building registry, apartment management, common expense tracking (κοινοχρηστα), support ticketing, polls, professional directory, calendar, and notifications
- **Target Users:** Property managers, building managers, owners, tenants, property agents, administrators, and service professionals

2. Project Overview

Project Name	Building Manager Network
Type	Full-Stack Web Application (Backend-Focused)
Backend Base URL	`http://localhost:8080/api/v1/`
Frontend	Angular (port 4200, separate repository)
API Documentation	`http://localhost:8080/api/v1/swagger-ui.html`
Primary Language	Java 21
Framework	Spring Boot 3.2.5
Database	PostgreSQL
Build System	Maven
Containerization	Docker & Docker Compose
Source Files	~180 Java files

Business Problem

Multi-tenant building management in Greece (and similar markets) involves complex workflows: tracking monthly common expenses (κοινοχρηστα), allocating costs across apartments based on ownership shares, managing maintenance requests, organizing building-wide votes, and maintaining communication between owners, tenants, and managers. Traditional approaches rely on paper, spreadsheets, and fragmented communication channels.

Solution

A centralized digital platform offering:

- Unified building and apartment registry with detailed attributes
- Automated common expense statement generation with itemized per-apartment allocation
- Online payment tracking and collection rate monitoring
- Role-based access ensuring data privacy and appropriate permissions
- Email-based invitation system for onboarding stakeholders

- Integrated support ticket system for maintenance and issue reporting
- Professional business directory connecting buildings with service providers
- Built-in polling and calendar features for community decisions and events

3. Technology Stack

Backend Technologies

Technology	Version	Purpose
Java	21	Primary programming language
Spring Boot	3.2.5	Application framework (DI, MVC, data access, security)
Spring Data JPA / Hibernate	(via Boot)	ORM, database access, repository pattern
Spring Security	(via Boot)	Authentication & authorization
Spring Mail	(via Boot)	Email notifications
Spring Validation	(via Boot)	Bean validation (Jakarta Validation)
PostgreSQL	15 (Alpine)	Primary database
JWT (jjwt)	0.11.5	Stateless token-based authentication
Lombok	1.18.30	Boilerplate reduction (@Data, @Builder, etc.)
Thymeleaf	(via Boot)	HTML email template rendering
Springdoc OpenAPI	2.3.0	Swagger UI / OpenAPI specification
Jakarta Mail	(via Boot)	Email sending
JAXB	4.0.x	XML binding for JWT dependencies

Infrastructure & DevOps

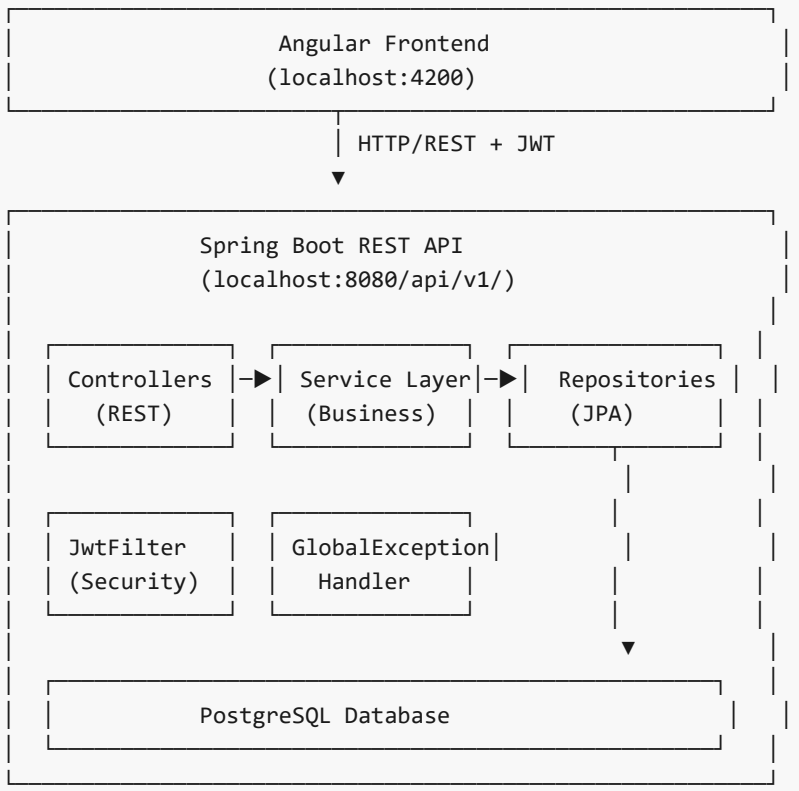
Tool	Purpose
Maven	Build automation & dependency management
Docker	Containerization (multi-stage build)
Docker Compose	Orchestration (app + PostgreSQL + MailDev)
MailDev	Development email server (Web UI on port 1080)

Frontend (External Repository)

The frontend is built with **Angular**, communicating with the backend via REST API. CORS is configured to allow requests from `http://localhost:4200`.

4. System Architecture

High-Level Architecture



Application Layers

Layer	Package(s)	Responsibility
Controller	`*.controller`	HTTP request handling, input validation, response formatting
Service	`*.service`	Business logic, orchestration, transaction management
Repository	`*.repository`	Database access via Spring Data JPA
Entity	`*.entity` (or main package)	JPA domain models with relationships
DTO	`*.dto` (or nested)	Data transfer objects for API communication
Mapper	`*.mapper`	Entity ↔ DTO mapping

Security	`security/`	JWT filtering, authentication, authorization
Config	`config/`	Spring configuration, OpenAPI, CORS, auditing
Handler	`handler/`	Global exception handling, standardized error responses

Data Flow Example: Common Expense Statement Creation

1. Authenticated user (with MANAGER/FULL permission) sends POST request to create a statement for a building
2. `JwtFilter` validates JWT token and sets authentication context
3. `CommonExpenseStatementController` receives request, validates permissions via `BuildingPermissionService`
4. `CommonExpenseStatementService` orchestrates: creates statement, generates items, calculates per-apartment allocations based on ownership thousandths
5. `CommonExpenseStatementRepository` persists the statement entity
6. Email notifications sent to affected residents/owners via `EmailService`
7. Response DTO returned to frontend

5. Core Business Modules

5.1 Authentication & User Management

> **Packages:** `auth/` , `user/` , `security/` , `token/` , `role/`

Features

- User registration with email-based 6-digit activation code
- JWT authentication (30-minute token expiry)
- Password reset flow with secure token
- 8 pre-defined roles seeded at application startup
- Profile image upload and management
- User profile retrieval and update

Authentication Flow

```
Registration:
  User → POST /auth/register → Email sent (6-digit code) → User verifies via POST /

Login:
  User → POST /auth/authenticate → JWT returned → Subsequent requests include "Bear

Password Reset:
  User → POST /auth/forgot-password → Email with reset token → POST /auth/reset-pas
```

5.2 Building & Apartment Management

> **Packages:** `building/` , `apartment/` , `buildingMember/` , `buildingDocument/`

Building Features

- Full CRUD with detailed attributes: address, floors, heating type, parking, storage, common spaces
- Unique building code for easy join-by-code functionality
- Assignment to Property Manager (company) and Building Manager
- Support for self-managed and company-managed buildings

- Document upload with categorized storage
- Active/enable status management

Apartment Features

- CRUD with owner, resident, parking, storage, and percentage allocation fields
- Batch creation for rapid building setup
- Percentage-based cost allocation (commonPercent, elevatorPercent, heatingPercent)
- Assignment to owners and residents (users)
- Support for manager house designation

5.3 Common Expense Management (Κοινοχρηστα)

> **Packages:** `commonExpenseStatement/`, `commonExpenseItem/`, `commonExpenseAllocation/`
> **Business Value:** This is the core revenue-generating feature of the platform.

Three-Tier Expense Model

Level	Entity	Description
1	CommonExpenseStatement	Monthly statement per building (code, month year, subtotal, total, tax %, discount %, status)
2	CommonExpenseItem	Line items within a statement (category: COMMON, HEATING, ELEVATOR, EQUAL, OWNERS; price, description)
3	CommonExpenseAllocation	Per-apartment allocation with amount, paid amount, due date, payment reference, ownership share snapshot

Expense Categories

- **COMMON:** General building expenses (cleaning, lighting, etc.)
- **HEATING:** Central heating fuel costs
- **ELEVATOR:** Elevator maintenance and repairs
- **EQUAL:** Equal-split expenses
- **OWNERS:** Expenses borne only by property owners (not tenants)

Business Rules

- Allocations are calculated based on apartment ownership thousandths (commonPercent, elevatorPercent, heatingPercent)
- Different expense categories can be allocated differently (e.g., heating by heating percent, elevator by elevator percent)
- OWNERS category expenses are assigned only to owners, not residents
- Each statement can be saved as a draft before finalizing
- Paid amounts tracked per allocation with payment date and method

5.4 Payment Tracking

> **Package:** `payment/`

Tracks payments made against common expense allocations. Supports three payment methods: Cash, Bank Transfer, Card. Provides building-level summaries, current-month collection status, and per-statement reconciliation.

5.5 Invite System

> **Package:** `invite/`

Invite Types

- **Apartment Invites:** Assign users as Owner, Resident, or Building Manager to specific apartments
- **Property Agent Invites:** Link agents to companies for multi-property management
- **Admin Agent Invites:** Admin-level agent role

Invite Flow

1. Authorized user creates invite with email, role, and property reference
2. System sends email containing a unique UUID token link
3. Recipient clicks link, authenticates (or registers), and accepts the invite
4. System validates token, updates user role, creates building membership, and assigns apartment
5. Token expires after 7 days if not accepted

5.6 Support Ticket System

> **Package:** `supportTicket/`

Ticket Model

- **Statuses:** OPEN → IN_PROGRESS → RESOLVED → CLOSED | CANCELLED
- **Priorities:** LOW, MEDIUM, HIGH, URGENT
- **Categories:** MAINTENANCE, CLEANING, ELECTRICAL, PLUMBING, HEATING, ELEVATOR, NOISE, OTHER
- **Target Roles:** BUILDING_MANAGER, PROPERTY_MANAGER, PROPERTY_AGENT, ADMIN

Features

- Ticket creation with category, priority, and target role routing
- Agent assignment and reassignment
- Multi-threaded comments with comment types
- Role-based inbox views
- Status workflow with validation

5.7 Professional Business Directory

> **Package:** `professional/`
> Sub-packages: `professionalFavorite`, `professionalImage`, `professionalPartner`, `professionalReview`

Categories

ELECTRICIAN, PLUMBER, ELEVATOR_TECHNICIAN, CLEANING_SERVICE, HEATING_TECHNICIAN, LOCKSMITH, AIR_CONDITION_TECHNICIAN, PAINTER, PEST_CONTROL, GENERAL_REPAIRS, OTHER

Features

- Business profile with contact info, address, and description
- Multiple image upload per business
- Rating and review system
- Favorite/bookmark functionality for users

- Partner/multi-business linking
- Verified and active status flags
- Admin statistics (total businesses, verified, by category)

5.8 Polls & Voting

> **Package:** `poll/`

Building-level polls with multiple options and start/end dates. Residents and owners can cast votes. Results tracked per option and per user.

5.9 Calendar & Events

> **Package:** `calendar/`

Per-building calendar events with title, description, date range, color coding, and pinning for important announcements.

5.10 Notifications

> **Package:** `notification/`

In-app notification system with type, JSON payload, read/unread status, and user targeting. Used for expense statements, ticket updates, invite notifications, and other system events.

5.11 Dashboards & Analytics

> **Packages:** `adminDashboard/` , `pmDashboard/` , `managerDashboard/` , `userDashboard/`

Dashboard	Audience	Key Metrics
Admin Dashboard	Platform administrators	Total buildings/users, growth trends, user activity, engagement stats, operational issues, platform overview

Property Manager Dashboard	Property management companies	Company building portfolio, expense collection rates, financial charts, membership stats, pending items, invite stats, activity feed
Building Manager Dashboard	Individual building managers	Monthly expense amounts, collection stats, building-level financials
User Dashboard	Residents & owners	Pending statements, payment history, mini-charts of expense trends, user statement summaries

6. Data Model & Entity Relationships

Core Entity Diagram

```
User ──┬── Role (M:1)
       ├── BuildingMember (1:M) ── Building (M:1)
       ├── Building (as manager, 1:M)
       ├── Building (as propertyManager, 1:M)
       ├── Apartment (as resident/owner, 1:M)
       ├── Token (1:M)
       ├── Payment (1:M) ── CommonExpenseAllocation (M:1)
       ├── Notification (1:M)
       ├── SupportTicket (as creator/agent, 1:M)
       ├── Company (M:1)
       ├── ProfessionalBusiness (1:M)
       └── ProfessionalFavorite (1:M)

Building ──┬── Apartment (1:M)
           ├── BuildingMember (1:M)
           ├── CommonExpenseStatement (1:M)
           ├── Calendar (1:M)
           ├── Poll (1:M)
           ├── SupportTicket (1:M)
           ├── BuildingNotificationSettings (1:M)
           ├── BuildingDocument (1:M)
           ├── Company (M:1)
           └── UserBuildingPermission (1:M)

Apartment ──┬── CommonExpenseAllocation (1:M)
            ├── BuildingMember (1:M)
            └── Invite (1:M)

CommonExpenseStatement ──┬── CommonExpenseItem (1:M)
                         └── CommonExpenseAllocation (1:M)

CommonExpenseItem ── CommonExpenseAllocation (1:M)

SupportTicket ── TicketComment (1:M)

Poll ── PollOption (1:M)
PollOption ── Vote (1:M)
```

Key Entity: Building

Field	Type	Description
buildingCode	String	Unique code for join-by-code functionality

name	String	Building name
street1, stNumber1, street2, stNumber2	String	Address lines
city, state, region, postalCode, country	String	Location
floors, apartmentsNum	Integer	Building specifications
sqMetersTotal, sqMetersCommonSpaces	double	Area measurements
heatingType	Enum	OIL, NATURAL_GAS, ELECTRIC, HEAT_PUMP, NONE
heatingCapacityLitres	Double	Heating fuel capacity
parkingExist, parkingSpacesNum	boolean/Integer	Parking information
undergroundFloorExist, halfFloorExist, etc.	boolean	Additional floor specifications
active, enable	boolean	Status flags
propertyManager	@ManyToOne → User	Property management company contact
manager	@ManyToOne → User	Building manager
company	@ManyToOne → Company	Managing company

Key Entity: Apartment

Field	Type	Description
ownerFirstName, ownerLastName	String	Owner name (can be pre-registration)
number, sqMetersApart, floor	String	Apartment identifiers
parkingSpace, isRented	Boolean	Status flags
residentFirstName, residentLastName	String	Resident name

commonPercent, elevatorPercent, heatingPercent	Double	Cost allocation percentages (thousandths)
building	@ManyToOne → Building	Parent building
resident, owner	@ManyToOne → User	Linked user accounts

7. API Endpoints Reference

Authentication

Method	Endpoint	Description
POST	`/auth/register`	Register new user
POST	`/auth/authenticate`	Login, returns JWT
POST	`/auth/activate-account`	Activate with 6-digit code
POST	`/auth/forgot-password`	Request password reset
POST	`/auth/reset-password`	Reset password with token
GET	`/auth/me`	Get current user profile

Buildings

Method	Endpoint	Description
POST	`/buildings`	Create building
GET	`/buildings/{id}`	Get building by ID
GET	`/buildings`	List buildings (paginated)
PUT	`/buildings/update/{id}`	Update building
DELETE	`/buildings/{id}`	Delete building
POST	`/buildings/join-by-code`	Join building via code
POST	`/buildings/self`	Create self-managed building
POST	`/buildings/company`	Create company-managed building
GET	`/buildings/manager`	Manager's buildings

Apartments

Method	Endpoint	Description

POST	`/apartments`	Create apartment
POST	`/apartments/batch`	Batch create apartments
GET	`/apartments/building/{id}`	List by building (paginated)
PUT	`/apartments/{id}`	Update apartment
DELETE	`/apartments/delete/{id}`	Delete apartment
GET	`/apartments/my-apartments`	User's apartments

Common Expenses

Method	Endpoint	Description
POST	`/expenses/statements/{buildingId}/createAndSend`	Create & send statement
POST	`/expenses/statements/{buildingId}/draft`	Save as draft
GET	`/expenses/statements/{id}`	Get statement
PUT	`/expenses/statements/{id}`	Update statement
DELETE	`/expenses/statements/{id}`	Delete statement

Support Tickets

Method	Endpoint	Description
POST	`/support-tickets`	Create ticket
GET	`/support-tickets/myTickets`	User's tickets
GET	`/support-tickets/inbox`	Agent's inbox
PATCH	`/support-tickets/{id}/status`	Update status
PATCH	`/support-tickets/{id}/assign-agent`	Assign agent
POST	`/support-tickets/{id}/comments`	Add comment

Polls

Method	Endpoint	Description
POST	`/polls`	Create poll
GET	`/polls/building/{buildingId}`	Building polls
POST	`/polls/{pollId}/vote`	Cast vote

Professional Directory

Method	Endpoint	Description
POST	`/professionals`	Create business listing
GET	`/professionals`	Search businesses
GET	`/professionals/{id}`	Get business details
POST	`/professionals/{id}/reviews`	Add review
POST	`/professionals/{id}/favorite`	Toggle favorite

Dashboards

Method	Endpoint	Description
GET	`/admin/dashboard`	Admin platform overview
GET	`/pm/dashboard`	Property Manager dashboard
GET	`/dashboard/building/{buildingId}`	Building Manager dashboard
GET	`/dashboard/user`	User dashboard

8. Role-Based Access Control

Role Hierarchy

Role	Level	Description
Admin	System-wide	Full platform access, all buildings, user management, admin dashboard
PropertyManager	Company-wide	Manages all buildings under their company, PM dashboard, agent management
BuildingManager	Building-level	Manages specific building(s), expense statements, tickets, polls
PropertyAgent	Company-wide	Support ticket agent, assigned to company buildings
AdminAgent	System-wide	Support ticket agent with cross-building access
Owner	Apartment-level	Owns apartment(s), views expenses, pays owner-specific charges
Resident	Apartment-level	Resides in apartment, views expenses, pays resident charges
User	Base	Default role after registration, limited to profile management

Permission Model

The system uses a multi-layered permission model:

1. **Role-Based:** Each user has a primary role (from Role entity)
2. **Building Membership:** BuildingMember entity links users to buildings with status (JOINED, PENDING, REMOVED)
3. **Explicit Permissions:** UserBuildingPermission entity provides granular per-building permissions (VIEW, MANAGE, FULL)

Permission Resolution

```
BuildingPermissionService.canViewBuilding(user, building):
  1. If user has ADMIN role → ALLOW
  2. If user has explicit FULL/MANAGE permission on building → ALLOW
```

```
3. If user is building manager → ALLOW
4. If user is building member → ALLOW
5. Otherwise → DENY
```

9. Deployment & DevOps

Docker Compose Architecture

```
docker-compose.yml:
┌──────────────────────────────────────────────────────┐
│  Service: postgres                                   │
│  Image: postgres:15-alpine                           │
│  Port: 5432                                          │
│  Volume: postgres (persistent data)                  │
│  Health: pg_isready                                  │
│  Environment: DB credentials                         │
├──────────────────────────────────────────────────────┤
│  Service: mail-dev                                   │
│  Image: maildev/maildev                              │
│  Ports: 1080 (Web UI), 1025 (SMTP)                   │
├──────────────────────────────────────────────────────┤
│  Service: app                                        │
│  Build: multi-stage Dockerfile                       │
│  Port: 8080                                          │
│  Depends: postgres (healthy), mail-dev               │
│  Health: curl /actuator/health                       │
└──────────────────────────────────────────────────────┘
```

Docker Build (Multi-Stage)

Stage	Base Image	Purpose
Build	maven:3.8.4-eclipse-temurin-21	Compile & package with Maven
Runtime	eclipse-temurin:21-jre	Run the Spring Boot JAR

Configuration

- `application.yml` — Default configuration (DB, mail, JWT, file storage, CORS)
- `application-dev.yml` — Development overrides
- Database: PostgreSQL with connection pooling
- File uploads stored locally under `/uploads/`
- JWT secret configurable via environment variable
- MailDev for email testing in development

10. Test Coverage & Quality Assurance

Current State

> ⚠ **Critical Gap:** Test coverage is minimal. Only 1 test exists — a basic Spring Boot context-loading smoke test.

Test Type	Count	Coverage
Unit Tests	0	0%
Integration Tests	0	0%
Repository Tests	0	0%
Controller Tests	0	0%
Context Load Test	1	N/A

Recommendations

- Implement unit tests for all service classes (JUnit 5 + Mockito)
- Add repository integration tests with @DataJpaTest and testcontainers for PostgreSQL
- Add controller tests with @WebMvcTest for REST endpoints
- Aim for >70% code coverage on core business logic (especially expense allocation, invite processing, permission checking)
- Introduce API-level integration tests with TestRestTemplate or REST Assured
- Consider adding archUnit tests for architectural rule enforcement

11. Business Value & Market Position

Target Market

- **Primary:** Greek multi-tenant residential buildings (πολυκατοικίες) — estimated 500,000+ buildings
- **Secondary:** Property management companies managing multiple buildings
- **Tertiary:** Commercial property management, residential complexes, housing cooperatives

Revenue Model Potential

Model	Description
SaaS Subscription	Monthly/annual fee per building or per property manager
Professional Directory	Featured listings, verified badges, lead fees
Payment Processing	Transaction fee on digital payments (if in-app payments integrated)
Premium Features	Advanced analytics, bulk SMS, document storage, API access

Competitive Advantages

- **Comprehensive:** Covers the full building management lifecycle in one platform
- **Role-Specific:** Designed for the Greek market with proper distinction between owners, residents, and managers
- **Invite-Based Onboarding:** Low friction for adding stakeholders
- **Community Features:** Polls, calendar, professional directory — creates an ecosystem, not just a utility

Key Business Metrics Tracked

- Number of registered buildings and apartments
- User engagement (activity metrics)
- Expense collection rates per building
- Pending vs. completed payments

- Support ticket volume and resolution times
- Invite acceptance rates
- Professional directory listing growth

12. Recommendations & Next Steps

Critical (High Priority)

1. **Test Coverage:** Implement comprehensive testing (unit, integration, API) — this is essential for production readiness
2. **Error Handling:** Review and standardize error responses across all endpoints
3. **API Versioning:** Implement versioning strategy (URL or header-based) for future backward compatibility

Important (Medium Priority)

1. **Monitoring & Observability:** Add structured logging (SLF4J), metrics (Micrometer), and tracing
2. **Audit Trail:** Enhance existing auditing to track all critical operations (expense creation, permission changes)
3. **Rate Limiting:** Implement API rate limiting to prevent abuse
4. **Pagination Standardization:** Ensure consistent pagination across all list endpoints
5. **Email Template Enhancement:** Add more templates for various notification types

Future Enhancements (Low Priority / Roadmap)

1. **Multi-Language Support:** i18n for Greek, English, and other languages
2. **Mobile App:** React Native or Flutter application for on-the-go access
3. **In-App Payments:** Integrate with payment gateways (stripe, viva wallet) for digital collections
4. **Smart Integration:** IoT sensor data for heating monitoring, water leak detection
5. **Document OCR:** Scan and digitize paper utility bills for automatic expense entry
6. **Chat System:** Real-time messaging between building stakeholders
7. **Maintenance Marketplace:** Enhanced professional directory with quoting/bidding workflows

Security Review Items

Area	Status	Recommendation

JWT Secret Management	Environment variable	Consider using Vault or AWS Secrets Manager for production
Password Policy	Basic validation	Enforce complexity requirements (upper, lower, digit, special)
API Input Validation	Jakarta Validation	Add custom validators for domain-specific fields (AFM, building codes)
CORS Configuration	Localhost only	Restrict to specific production domain(s)
SQL Injection	JPA/Hibernate	Verify all native queries use parameterized inputs

Technical Debt

- Test file located in legacy package `com.building` instead of `com.buildingmanager`
- Lombok widely used — consider adding delombok step for debugging/profiling
- No API versioning strategy defined
- No CI/CD pipeline configuration (GitHub Actions, Jenkins)
- File storage uses local filesystem — consider cloud storage (S3, GCS) for production

---